ПРАКТИЧЕСКАЯ РАБОТА № 3

Модульное тестирование класса AuthManager средствами pytest

1 ЦЕЛЬ РАБОТЫ

Научиться писать модульные тесты на pytest для класса `AuthManager`, управляющего пользователями, аутентификацией, подсчётом пользователей по странам и переводом средств. В тестах необходимо продемонстрировать базовые (3), параметризованные (3) и проверяющие исключения (2) тесты, фикстуру базы данных, не менее двух меток, обязательный тест на SQL-инъекции, один пропускаемый тест и запуск с ограничением числа неудач `--maxfail=2`.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

pytest — фреймворк модульного тестирования для Python. Тест — это функция с префиксом `test_`, в которой используются обычные операторы `assert`. pytest сам обнаруживает тестовые файлы, выполняет функции и формирует отчёт. Ключевые возможности, применённые в работе, перечислены ниже.

Таблица 1 – Возможности pytest, использованные в работе

Возможность	Назначение	Применение в работе
Фикстуры (`@pytest.fixture`)	Подготовка и очистка тестового окружения; результат подставляется в тест по имени.	Соединение SQLite `:memory:`, экземпляр `AuthManager`, набор из трёх пользователей.
Параметризация (`@pytest.mark.parametrize`)	Один тест выполняется для набора входных данных; каждый набор — отдельный результат.	Подсчёт по странам, варианты аутентификации, суммы перевода.
Метки (`@pytest.mark.*`)	Классификация тестов и выборочный запуск через `-m`.	Метки `basic`, `exception`, `security`, `parametrized`.
Проверка исключений (`pytest.raises`)	Утверждение, что операция завершается ожидаемым исключением.	Недостаток средств и перевод от несуществующего пользователя.
Пропуск (`@pytest.mark.skip`)	Тест документируется, но не выполняется.	Проверка версии Python.

Возможность	Назначение	Применение в работе
`--maxfail=N`	Остановка прогона после N неудач — экономит время при массовых падениях.	Демонстрация на файле `test_failure_demo.py` с `--maxfail=2`.

3 ТЕСТИРУЕМЫЙ КЛАСС AUTHMANAGER

Класс из задания хранит пользователей в таблице SQLite (`users`: id, username, password, country, balance), умеет регистрировать пользователя, аутентифицировать его, удалять, искать по идентификатору, считать пользователей по стране и переводить средства. Код класса приведён ниже без изменений — тестируется именно выданная реализация.

```
import sqlite3


class AuthManager:
    def __init__(self, connection):
        self.connection = connection
        self.create_tables()

    def create_tables(self):
        with self.connection:
            self.connection.execute("""
                CREATE TABLE IF NOT EXISTS users (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    username TEXT NOT NULL UNIQUE,
                    password TEXT NOT NULL,
                    country TEXT NOT NULL,
                    balance REAL NOT NULL
                )
            """)

    def register_user(self, username, password, country, balance):
        with self.connection:
            self.connection.execute("""
                INSERT INTO users (username, password, country, balance)
                VALUES ('{}', '{}', '{}', {})
            """.format(username, password, country, balance))  # Уязвимость SQL-инъекции

    def authenticate_user(self, username, password):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users
            WHERE username = '{}' AND password = '{}'
        """.format(username, password))  # Уязвимость SQL-инъекции
        return cursor.fetchone()

    def delete_user(self, user_id):
        with self.connection:
            self.connection.execute("""
                DELETE FROM users WHERE id = {}
            """.format(user_id))  # Уязвимость SQL-инъекции

    def get_user_by_id(self, user_id):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users WHERE id = {}
        """.format(user_id))  # Уязвимость SQL-инъекции
        return cursor.fetchone()

    def count_users_by_country(self, country):
        cursor = self.connection.cursor()
```

```
        cursor.execute("""
            SELECT COUNT(*) FROM users WHERE country = '{}'
        """.format(country))  # Уязвимость SQL-инъекции
        return cursor.fetchone()[0]

    def transfer_balance(self, from_user_id, to_user_id, amount):
        with self.connection:
            # Проверяем, достаточно ли средств
            cursor = self.connection.cursor()
            cursor.execute("SELECT balance FROM users WHERE id =
{}".format(from_user_id))  # Уязвимость SQL-инъекции
            from_balance = cursor.fetchone()[0]

            if from_balance < amount:
                raise ValueError("Insufficient funds")

            # Выполняем перевод
            self.connection.execute("""
                UPDATE users SET balance = balance - {} WHERE id = {}
            """.format(amount, from_user_id))  # Уязвимость SQL-инъекции
            self.connection.execute("""
                UPDATE users SET balance = balance + {} WHERE id = {}
            """.format(amount, to_user_id))  # Уязвимость SQL-инъекции
```

Важно. В классе используются SQL-запросы, собранные через `.format()`, а пароли хранятся в открытом виде. Это осознанные дефекты исходного кода: тесты должны их обнаружить и задокументировать, а не скрыть. Исправление выходит за рамки задания, но рекомендации приведены в разделе 9.

4 СТРУКТУРА ТЕСТОВОГО НАБОРА

Тесты находятся в файле `test_auth_manager.py`. Всего собирается 19 тестовых случаев: 17 проходят, 1 пропускается, 1 помечен как ожидаемо падающий (xfail). Соответствие требованиям задания приведено в таблице.

Таблица 2 – Соответствие требованиям задания

Требование	Реализация	Количество
Базовые тесты	`test_register_and_get_user`, `test_authenticate_user_success`, `test_delete_user`	3
Параметризованные тесты	`test_count_users_by_country` (3 набора), `test_authenticate_user_parametrized` (4), `test_transfer_balance_parametrized` (3)	3 теста / 10 случаев
Тестирование исключений	`test_transfer_insufficient_funds_raises`, `test_transfer_from_unknown_user_raises`	2
Фикстура базы данных	db — SQLite `:memory:`; производные `auth_manager` и `users`	3 фикстуры
Метки	`basic`, `exception`, `security`, `parametrized`	4
Тест на SQL-инъекции	`test_sql_injection_authentication_bypass`, `test_sql_injection_count_by_country`, `test_sql_injection_must_be_blocked`	3
Пропускаемый тест	`test_python_version` с `@pytest.mark.skip`	1

Требование	Реализация	Количество
Запуск с `--maxfail=2`	команда в разделе 8 и снимок экрана	—

5 ФИКСТУРЫ И БАЗА ДАННЫХ

Для изоляции тестов используется временная база SQLite в оперативной памяти. Каждый тест получает новое соединение, поэтому данные одного теста не влияют на другой. Фикстура `auth_manager` создаёт экземпляр класса поверх этого соединения и автоматически создаёт таблицы, а `users` наполняет базу тремя пользователями для параметризованных проверок.

```
@pytest.fixture
def db():
    """Временная БД SQLite в оперативной памяти."""
    connection = sqlite3.connect(":memory:")
    yield connection
    connection.close()


@pytest.fixture
def auth_manager(db):
    """Экземпляр AuthManager, привязанный к тестовой БД."""
    return AuthManager(db)


@pytest.fixture
def users(auth_manager):
    """Три пользователя для параметризованных тестов."""
    auth_manager.register_user("user1", "password123", "CountryA", 1000)
    auth_manager.register_user("user2", "password456", "CountryA", 500)
    auth_manager.register_user("user3", "password789", "CountryB", 250)
    return auth_manager
```

Благодаря `yield` соединение закрывается после теста даже при падении проверки. Это исключает утечку ресурсов и «залипание» состояния между тестами.

6 БАЗОВЫЕ ТЕСТЫ

Три базовых теста проверяют основной жизненный цикл пользователя: регистрацию и чтение, успешную аутентификацию и удаление. Все они помечены `@pytest.mark.basic`.

```
@pytest.mark.basic
def test_register_and_get_user(auth_manager):
    auth_manager.register_user("alice", "secret", "Russia", 1500)
    user = auth_manager.get_user_by_id(1)

    assert user is not None
    assert user[1] == "alice"
    assert user[2] == "secret"
    assert user[3] == "Russia"
```

```
    assert user[4] == 1500


@pytest.mark.basic
def test_authenticate_user_success(auth_manager):
    auth_manager.register_user("bob", "qwerty", "Russia", 100)
    user = auth_manager.authenticate_user("bob", "qwerty")
    assert user is not None
    assert user[1] == "bob"


@pytest.mark.basic
def test_delete_user(auth_manager):
    auth_manager.register_user("carol", "pass", "Belarus", 300)
    auth_manager.delete_user(1)
    assert auth_manager.get_user_by_id(1) is None
```

7 ПАРАМЕТРИЗОВАННЫЕ ТЕСТЫ

Параметризация позволяет проверить одну и ту же логику на нескольких наборах данных. Первый тест — подсчёт пользователей по стране (пример из задания), второй — варианты аутентификации, третий — переводы разных сумм с проверкой обоих балансов.

```
@pytest.mark.parametrize(
    "country, expected_count",
    [("CountryA", 2), ("CountryB", 1), ("CountryC", 0)],
)
def test_count_users_by_country(users, country, expected_count):
    assert users.count_users_by_country(country) == expected_count


@pytest.mark.parametrize(
    "username, password, expected_result",
    [
        ("user1", "password123", True),
        ("user1", "wrong-password", False),
        ("unknown", "password123", False),
        ("", "", False),
    ],
)
def test_authenticate_user_parametrized(users, username, password, expected_result):
    user = users.authenticate_user(username, password)
    assert (user is not None) is expected_result


@pytest.mark.parametrize(
    "amount, expected_from, expected_to",
    [(100, 900, 600), (500, 500, 1000), (1000, 0, 1500)],
)
def test_transfer_balance_parametrized(auth_manager, amount, expected_from,
expected_to):
    auth_manager.register_user("sender", "p1", "CountryA", 1000)
    auth_manager.register_user("receiver", "p2", "CountryB", 500)
    auth_manager.transfer_balance(1, 2, amount)

    assert auth_manager.get_user_by_id(1)[4] == expected_from
    assert auth_manager.get_user_by_id(2)[4] == expected_to
```

Наборы включают граничный случай `amount = 1000` (перевод всего баланса, остаток 0) и `amount = 500` (половина баланса). Пустые логин и пароль проверяют, что аутентификация не превращается в «пропуск любого».

8 ТЕСТИРОВАНИЕ ИСКЛЮЧЕНИЙ

Проверяются два ошибочных сценария перевода средств. Первый — недостаток средств: класс должен выбросить `ValueError` с сообщением «Insufficient funds». Второй — перевод от несуществующего пользователя: класс не обрабатывает отсутствие записи явно, поэтому `cursor.fetchone()` возвращает `None`, и при обращении `[0]` возникает `TypeError`. Тест фиксирует фактическое поведение и документирует дефект.

```
@pytest.mark.exception
def test_transfer_insufficient_funds_raises(auth_manager):
    auth_manager.register_user("poor", "p1", "CountryA", 100)
    auth_manager.register_user("rich", "p2", "CountryB", 1000)

    with pytest.raises(ValueError, match="Insufficient funds"):
        auth_manager.transfer_balance(1, 2, 500)


@pytest.mark.exception
def test_transfer_from_unknown_user_raises(auth_manager):
    """Документируем дефект: отсутствие пользователя не обработано явно."""
    auth_manager.register_user("receiver", "p2", "CountryB", 1000)

    with pytest.raises(TypeError):
        auth_manager.transfer_balance(999, 1, 100)
```

Правильным поведением было бы явное исключение `ValueError("User not found")` или `KeyError`, а не `TypeError` из недр реализации. Тест зафиксирован как есть, чтобы после исправления класса его можно было изменить осознанно.

9 ТЕСТЫ БЕЗОПАСНОСТИ: SQL-ИНЪЕКЦИИ

Обязательный тест на SQL-инъекции. Класс собирает запросы через `.format()`, поэтому пользовательский ввод попадает прямо в текст SQL. Тест 1 подставляет `admin' OR '1'='1` в поле имени пользователя и показывает, что вход выполняется без корректного пароля. Тест 2 подставляет `' OR '1'='1` в страну и получает всех пользователей вместо одного класса. Тест 3 описывает *требуемое безопасное* поведение и помечен `xfail(strict=True)`: на уязвимом коде он падает, а после исправления станет неожиданно проходить, и pytest об этом сообщит.

```
@pytest.mark.security
def test_sql_injection_authentication_bypass(auth_manager):
    """Инъекция в username позволяет войти без корректного пароля."""
    auth_manager.register_user("admin", "real-password", "Russia", 1000)

    injected_user = auth_manager.authenticate_user("admin' OR '1'='1", "wrong")

    assert injected_user is not None
    assert injected_user[1] == "admin"
```

```
@pytest.mark.security
def test_sql_injection_count_by_country(users):
    """Инъекция в country расширяет выборку на всех пользователей."""
    assert users.count_users_by_country("' OR '1'='1") == 3


@pytest.mark.security
@pytest.mark.xfail(reason="Ожидаемое безопасное поведение...", strict=True)
def test_sql_injection_must_be_blocked(auth_manager):
    auth_manager.register_user("admin", "real-password", "Russia", 1000)
    injected_user = auth_manager.authenticate_user("admin' OR '1'='1", "wrong")
    assert injected_user is None
```

Причина уязвимости — конкатенация строк. Запрос `SELECT * FROM users WHERE username = 'admin' OR '1'='1' AND password = 'wrong'` из-за приоритета `AND` над `OR` возвращает запись администратора независимо от пароля. Исправление: параметризованные запросы (`WHERE username = ? AND password = ?`) и хеширование паролей вместо хранения в открытом виде.

10 МЕТКИ, ПРОПУСК И КОНФИГУРАЦИЯ PYTEST

Метки объявлены в `pytest.ini`, чтобы pytest не выдавал предупреждения и позволял запускать группы тестов: `pytest -m basic`, `pytest -m security` и т. д. Один тест пропускается через `@pytest.mark.skip`: он документирует проверку версии Python и не влияет на результат прогона.

```
[pytest]
markers =
    basic: базовые позитивные тесты
    exception: тесты обработки исключений
    security: тесты безопасности и SQL-инъекций
    parametrized: параметризованные тесты

testpaths = .
python_files = test_*.py
addopts = -ra
```

```
@pytest.mark.skip(reason="Демонстрация пропуска теста: проверка версии Python")
def test_python_version():
    assert sys.version_info >= (3, 7)
```

Метки покрывают четыре группы: `basic` (3 теста), `exception` (2), `security` (3) и `parametrized`. Этого достаточно для требования «минимум две метки».

11 ЗАПУСК ТЕСТОВ И РЕЗУЛЬТАТЫ

Основной прогон выполняется командой:

```
python -m pytest test_auth_manager.py -v --maxfail=2 --tb=short
```

Ключи: `-v` — подробный вывод по каждому тесту, `--maxfail=2` — остановка после двух неудач, `--tb=short` — короткая трассировка. Результат корректного прогона: **17 passed, 1 skipped, 1 xfailed** за 0,17 с.

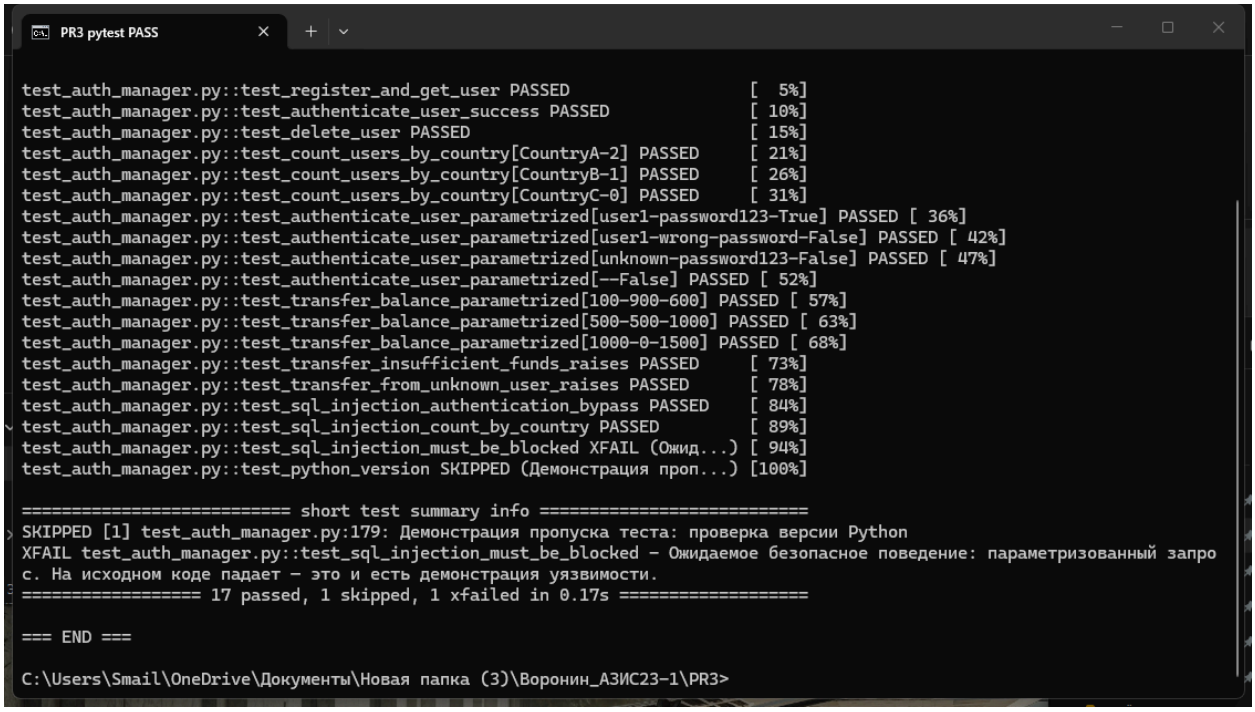


Рисунок 1 – Корректный прогон: 17 passed, 1 skipped, 1 xfailed

Таблица 3 – Запуск тестов и результаты

Группа	Тестов	Результат
Базовые (`basic`)	3	3 passed
Параметризованные	10 случаев	10 passed
Исключения (`exception`)	2	2 passed
Безопасность (`security`)	3	2 passed + 1 xfailed
Пропускаемый	1	1 skipped
Итого	**19**	**17 passed, 1 skipped, 1 xfailed**

11.1 Демонстрация неудачного прогона

Для подтверждения работы `--maxfail=2` подготовлен файл `test_failure_demo.py` с тремя тестами, два из которых намеренно содержат неверные ожидания: остаток после перевода 900 вместо 800 и количество пользователей 3 вместо 2. После второй неудачи pytest останавливает прогон, и третий тест не выполняется.

```
def test_wrong_expected_balance_after_transfer(auth_manager):
    ...
```

```
    assert manager.get_user_by_id(1)[4] == 900    # фактически 800.0


def test_wrong_expected_user_count(auth_manager):
    ...
    assert manager.count_users_by_country("CountryA") == 3    # фактически 2
```

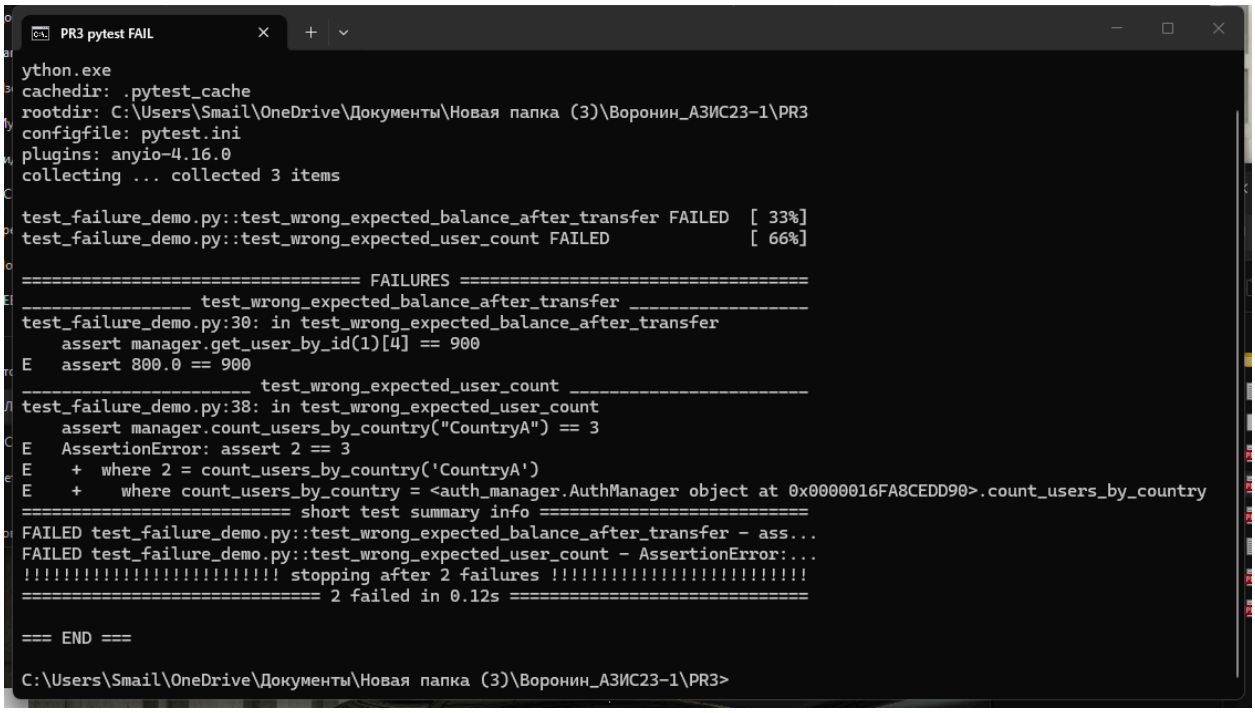


Рисунок 2 – Неудачный прогон: 2 failed, остановка после второй неудачи (--maxfail=2)

Вывод наглядно показывает оба типа диагностики: сравнение чисел (`assert 800.0 == 900`) и результат функции (`assert 2 == 3`). Строка `stopping after 2 failures` подтверждает срабатывание ограничения.

12 ВЫВОД

Разработан набор из 19 тестовых случаев для класса `AuthManager`. Реализованы три базовых теста, три параметризованных теста (10 наборов данных), два теста исключений, фикстура временной базы SQLite, четыре метки и один пропускаемый тест. Обязательный тест на SQL-инъекции подтвердил уязвимость: подстановка `admin' OR '1'='1` позволяет аутентифицироваться без пароля, а инъекция в страну расширяет выборку на всех пользователей. Дополнительно обнаружен дефект обработки несуществующего пользователя (`TypeError` вместо явного исключения). Прогон с `--maxfail=2` продемонстрирован на файле с намеренно неверными ожиданиями: после двух падений выполнение остановлено.

Рекомендации по исправлению класса: перейти на параметризованные запросы (`?`), хешировать пароли, явно обрабатывать отсутствие пользователя, обернуть перевод в транзакцию с проверкой обоих счетов и запретить отрицательные суммы перевода.

ПРИЛОЖЕНИЕ А. ПОЛНЫЙ КОД ТЕСТОВ

```
import sys

import pytest
import sqlite3

from auth_manager import AuthManager


# --------------------------------------------------------------------------
# Фикстуры
# --------------------------------------------------------------------------
@pytest.fixture
def db():
    """Временная БД SQLite в оперативной памяти."""
    connection = sqlite3.connect(":memory:")
    yield connection
    connection.close()


@pytest.fixture
def auth_manager(db):
    """Экземпляр AuthManager, привязанный к тестовой БД."""
    return AuthManager(db)


@pytest.fixture
def users(auth_manager):
    """Три пользователя для параметризованных тестов."""
    auth_manager.register_user("user1", "password123", "CountryA", 1000)
    auth_manager.register_user("user2", "password456", "CountryA", 500)
    auth_manager.register_user("user3", "password789", "CountryB", 250)
    return auth_manager


# --------------------------------------------------------------------------
# 1. Базовые тесты (3)
# --------------------------------------------------------------------------
@pytest.mark.basic
def test_register_and_get_user(auth_manager):
    auth_manager.register_user("alice", "secret", "Russia", 1500)
    user = auth_manager.get_user_by_id(1)

    assert user is not None
    assert user[1] == "alice"
    assert user[2] == "secret"
    assert user[3] == "Russia"
    assert user[4] == 1500


@pytest.mark.basic
def test_authenticate_user_success(auth_manager):
    auth_manager.register_user("bob", "qwerty", "Russia", 100)

    user = auth_manager.authenticate_user("bob", "qwerty")

    assert user is not None
    assert user[1] == "bob"


@pytest.mark.basic
def test_delete_user(auth_manager):
    auth_manager.register_user("carol", "pass", "Belarus", 300)
    auth_manager.delete_user(1)

    assert auth_manager.get_user_by_id(1) is None


# --------------------------------------------------------------------------
# 2. Параметризованные тесты (3)
```

```
# ----------------------------------------------------------------------
@pytest.mark.parametrize(
    "country, expected_count",
    [
        ("CountryA", 2),
        ("CountryB", 1),
        ("CountryC", 0),
    ],
)
def test_count_users_by_country(users, country, expected_count):
    assert users.count_users_by_country(country) == expected_count


@pytest.mark.parametrize(
    "username, password, expected_result",
    [
        ("user1", "password123", True),
        ("user1", "wrong-password", False),
        ("unknown", "password123", False),
        ("", "", False),
    ],
)
def test_authenticate_user_parametrized(users, username, password, expected_result):
    user = users.authenticate_user(username, password)
    assert (user is not None) is expected_result


@pytest.mark.parametrize(
    "amount, expected_from, expected_to",
    [
        (100, 900, 600),
        (500, 500, 1000),
        (1000, 0, 1500),
    ],
)
def test_transfer_balance_parametrized(auth_manager, amount, expected_from,
expected_to):
    auth_manager.register_user("sender", "p1", "CountryA", 1000)
    auth_manager.register_user("receiver", "p2", "CountryB", 500)

    auth_manager.transfer_balance(1, 2, amount)

    assert auth_manager.get_user_by_id(1)[4] == expected_from
    assert auth_manager.get_user_by_id(2)[4] == expected_to


# ----------------------------------------------------------------------
# 3. Тестирование исключений (2)
# ----------------------------------------------------------------------
@pytest.mark.exception
def test_transfer_insufficient_funds_raises(auth_manager):
    auth_manager.register_user("poor", "p1", "CountryA", 100)
    auth_manager.register_user("rich", "p2", "CountryB", 1000)

    with pytest.raises(ValueError, match="Insufficient funds"):
        auth_manager.transfer_balance(1, 2, 500)


@pytest.mark.exception
def test_transfer_from_unknown_user_raises(auth_manager):
    """Документируем дефект: отсутствие пользователя не обработано явно."""
    auth_manager.register_user("receiver", "p2", "CountryB", 1000)

    with pytest.raises(TypeError):
        auth_manager.transfer_balance(999, 1, 100)


# ----------------------------------------------------------------------
# 4. Тесты безопасности: SQL-инъекции (обязательный тест)
# ----------------------------------------------------------------------
@pytest.mark.security
def test_sql_injection_authentication_bypass(auth_manager):
    """Инъекция в поле username позволяет войти без корректного пароля.
```

```
    Тест проходит => уязвимость подтверждена. Это ожидаемое поведение
    исходного класса AuthManager, который выдаётся в задании.
    """
    auth_manager.register_user("admin", "real-password", "Russia", 1000)

    injected_user = auth_manager.authenticate_user("admin' OR '1'='1", "wrong")

    assert injected_user is not None
    assert injected_user[1] == "admin"


@pytest.mark.security
def test_sql_injection_count_by_country(users):
    """Инъекция в country расширяет выборку на всех пользователей."""
    injected_count = users.count_users_by_country("' OR '1'='1")

    assert injected_count == 3


@pytest.mark.security
@pytest.mark.xfail(
    reason="Ожидаемое безопасное поведение: параметризованный запрос. "
    "На исходном коде падает — это и есть демонстрация уязвимости.",
    strict=True,
)
def test_sql_injection_must_be_blocked(auth_manager):
    auth_manager.register_user("admin", "real-password", "Russia", 1000)

    injected_user = auth_manager.authenticate_user("admin' OR '1'='1", "wrong")

    assert injected_user is None


# ---------------------------------------------------------------------------
# 5. Пропускаемый тест
# ---------------------------------------------------------------------------
@pytest.mark.skip(reason="Демонстрация пропуска теста: проверка версии Python")
def test_python_version():
    assert sys.version_info >= (3, 7)
```